

## 3.3 Pointers

Sue has just finished working on her résumé and begins the arduous task of posting the update. She puts one copy on her LinkedIn page, another in her electronic portfolio, and another in the school's career site. Unfortunately, she also sent a number of copies to various prospective employers across the country. How can she update them? Rather than sending copies everywhere, it would have been much easier if she just sent links. This way, she would only have to update one location and, when people follow the link, they would also get the most recent version.

### Objectives

By the end of this class, you will be able to:

- Declare a pointer variable.
- Point to an existing variable in memory with a pointer variable.
- Get the data out of a pointer.
- Pass a pointer to a function.

### Prerequisites

Before reading this section, please make sure you are able to:

- Choose the best data type to represent your data (Chapter 1.2).
- Declare a variable (Chapter 1.2).
- Pass data into a function using both pass-by-value and pass-by-reference (Chapter 1.4).

## Overview

Up to this point, our variables have been working exclusively with data. This is familiar and straight-forward: if you want to make a copy of a value then use the assignment (=) operator. Often, however, it is more convenient to work with *addresses* rather than with data. Consider Sue's aforementioned scenario. If Sue would have distributed the address of her résumé (link to the document) rather than the data (the physical résumé), then the multiple-copy problem would not exist.

There are several reasons why working with addresses can be more convenient:

- **One master copy:** Often we want to keep one master copy of the data to avoid versioning and update problems like Sue's.
- **Size:** When working with large amounts of data (think arrays and strings), it can be expensive to make a copy of the data every time a function is called. For this reason, arrays are always passed by pointers to functions (more on that later).
- **References & citations:** Consider a lawyer citing a legal case. Rather than bringing the entire case into the courtroom to make a point, he instead cites the relevant case. This way, any interested party can go look at the original case if they are unsure of the details.

We use addresses in everyday life. Other names include links, references, citations, and pointers. For something to be considered a pointer, it must exhibit the following properties:

- **Points:** Each pointer must point to something. This could be data, a physical object, an idea, or whatever.
- **Mapping:** There must be a unique mapping. In other words, for every pointer, there must be exactly one object it is referring to.
- **Addressing scheme:** There must be some way to retrieve the data the pointer is referring to.

In a nutshell, a pointer is the address of data, rather than the data itself. For example, your debit card does not actually hold any money in it (data), instead it holds your account number (pointer to data). Today we will discuss the syntax of pointers and using pointers as parameters in functions. During the course of our discussion, we will also reveal a little secret of arrays: they are accessed using pointers!

## Syntax

Pointer syntax can be reduced to just three parts: declaring a pointer with the `*`, using the address-of operator (`&`) to derive an address of a variable, and the dereference operator (`*`) to retrieve the data from a given address.

```

{
    int speed = 65;
    int * pSpeed;

    pSpeed = &speed;

    cout << *pSpeed;
}
  
```

**speed**  
Every pointer needs to point to something. In this case, the pointer will point to `speed`.

**int \* pSpeed**  
The data-type of pointer is “pointer to an integer.” Notice that there are two parts to the declaration: the type it is pointing to “`int`” and the fact that it is a pointer “`*`”.  
Because the variable does not hold data but rather an address, it is helpful to name it differently. In this case `pSpeed` means “pointer to `speed`.”

**&speed**  
To get the address of `speed`, we use the address-of operator “`&`”. Since the data-type of `speed` is `int`, the data-type of `&speed` is “`int *`” or pointer to `int`.

**\*pSpeed**  
Use the dereference operator “`*`” to retrieve the data that `pSpeed` points to.

## Declaring a pointer

When declaring a normal data variable, it is necessary to specify the data-type. This is required so the compiler knows how to interpret the 1's and 0's in memory and how to evaluate expressions. Pointers are no different in this regard, but there is one extra degree of complexity. Not only is it necessary to specify the data-type of the data that is pointed to, it is also necessary to identify the fact that the variable is a pointer. Therefore, pointer declaration has two parts: the data-type and the `*`.

```
<data-type> * <pointer variable>;
```

The following is an example of a pointer to a float:

```
float * pGPA;
```

The first part of the declaration is the data-type we are pointing to (`float`). This is important because, after we dereference the pointer, the compiler needs to know what type of data we are working with.



### Sue's Tips

We need to remember to treat pointers differently than data variables because we need the dereference operator (\*) when accessing data. To help remember, always prefix a pointer variable with a p. This will make it less likely to confuse a pointer variable with a regular data variable.

## Getting the address of a variable

Every variable in C++ resides in some memory location. With the address-of operator (&), it is possible to query a variable for its address at any point in the program. The result is always an address, but the data-type depends on the type of the data being queried. Consider the following example:

```
{
    // a bunch of normal data variables
    int    valueInteger;           // integer
    float  valueFloatingPoint;    // floating point number
    bool   valueBoolean;         // Boolean
    char   valueCharacter;        // character

    // a bunch of pointer variables
    int    * pInteger;            // pointer to integer
    float  * pFloatingPoint;      // pointer to a floating point number
    bool   * pBoolean;           // pointer to a Boolean value
    char   * pCharacter;          // pointer to a character

    // assignments
    pInteger    = &valueInteger;    // both sides of = are pointers to integers
    pFloatingPoint = &valueFloatingPoint; // both sides are pointers to floats
    pBoolean     = &valueBoolean;    // both sides are pointers to Booleans
    pCharacter    = &valueCharacter; // both sides are pointers to characters
}
```

In the first assignment (`pInteger = &valueInteger`), the data-type of `valueInteger` is `int`. When the address-of operator is added to the expression, the data-type becomes “pointer to int.” The left-side of the assignment is `pInteger` which is declared as a “pointer to an int.” Since both sides are the same data-type (pointer to an int), then there is not a problem with the assignment. If we tried to assign `&valueInteger` to `pFloatingPoint`, we would get the following compile error:

```
example.cpp: In function “int main()”:
example.cpp:20: error: cannot convert “int*” to “float*” in assignment
```



All pointers are the same size, regardless of what they point to. This size is the native size of the computer. If, for example, you are working on a 32 bit computer, then a pointer will be four bytes in size:

```
assert(sizeof(int *) == 4);    // only true for 32 bit computers
assert(sizeof(char *) == 4);  // all addresses are the same size
```

However, if the computer was 64 bits, then the pointer would be eight bytes in size:

```
assert(sizeof(int *) == 8);    // only true for 64 bit computers
assert(sizeof(char *) == 8);  // again, what they point to does not matter
```

While this may seem counterintuitive, it is actually quite logical. The address to my college apartment in GPS coordinates was exactly the same size as the address to a nearby football stadium. The addresses were the same size, even though the thing they pointed to were not!

## Retrieving the data from a pointer

We can always retrieve the data from a given address using the dereference operator (\*). For this to be accomplished the compiler must know the data-type of the location in memory and the address must be valid. Since the data-type is part of the pointer declaration, the first part is not a problem. It is up to the programmer to ensure the second constraint is met. In other words, the compiler ensures that a data variable is always referring to a valid location in memory. However, with pointers, the programmer needs to set up the value. Consider the following example:

```
{
    int speed = 65;           // the location in memory we will be pointing to
    int * pSpeed;             // currently uninitialized. Don't dereference it!

    pSpeed = &speed;          // now it is initialized to the address of speed

    cout << *pSpeed << endl;  // need to use the * to get the data
}
```

If we removed the dereference operator (\*) from the cout statement: `cout << pSpeed << endl;`, then we would pass a “pointer to an integer” to cout. This would display not the value 65, but rather the location where that value exists in memory:

```
0x7fff9d235d74
```

If we skipped the initialization step in the above code (`pSpeed = &speed`), then the variable `pSpeed` would remain un-initialized. Thus, when we dereference it, it would refer to a location in memory (segment) the program does not own. This would cause a segmentation fault (a.k.a “crash”) at run-time:

```
Segmentation fault (core dumped)
```

### Example 3.3 – Variable vs. Pointer

Demo

This example will demonstrate the differences between working with pointers and working with variables. Pointers do not hold data so they can only be said to share values with other variables. Variables, on the other hand, make copies of values.

Solution

In the following example, we will have a standard variable called `account` which stores my current account balance. We will also have a pointer to my account called `pAccountNumber`. Observe how we add the ‘p’ prefix to pointer variables to remind us they are special. Finally, we will modify the account balance both through manipulating the `account` variable and the `pAccountNumber` variable.

```
{
    // Standard variable holding my account balance. I opened the account with
    // birthday money from granny (I love granny!).
    double account = 100.00;

    // Visiting the ATM, I get a receipt of my current account balance ($100.00)
    double receipt = account;

    // Pointer variable not currently pointing to anything. I have just declared
    // it, but it is still not initialized. We declare a pointer variable by
    // specifying the data type we are pointing to and the special '*' character
    double * pAccountNumber;

    // Now my account number refers to the variable account. We do this by
    // getting the address of the account variable. This is done with the
    // address-of operator '&'
    pAccountNumber = &account;

    // From here I can either access the account variable directly...
    account += 0.12; // interest from the bank

    // ... or I can access it through the pAccountNumber pointer. In this case, I
    // went to the ATM machine and added $20.00. Observe how I can access the
    // data of a pointer with the dereference operator '*'
    *pAccountNumber += 20.00;

    // Now I will display the differences
    cout << "Receipt:  $" << receipt << endl;    // the old value: $100.00
    cout << "Balance:  $" << account << endl;    // updated value: $120.12
}
```

The receipt is a standard variable, not changing to reflect the latest copy of the variable. My debit card contains my account number, a pointer! Thus going to the ATM machine (dereferencing the account number pointer) always gets me the latest copy of my account balance.

| account | pAccountNumber | receipt |
|---------|----------------|---------|
| 120.12  | ←              | 100.00  |

Challenge

As a challenge, change the value `receipt` to reflect a modest “adjustment” to your account.

```
receipt = 1000000.00;    // I wish this actually worked!
```

Notice how it does not influence the value in the variable `account`. Only pointers can do that!

See Also

The complete solution is available at [3-3-variableVsPointer.cpp](#) or:

```
/home/cs124/examples/3-3-variableVsPointer.cpp
```

Unit 3

|                                | Example 3.3 – Pointer to Master Copy                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |               |                 |           |               |                 |    |   |   |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|-----------------|-----------|---------------|-----------------|----|---|---|
| Demo                           | This example will demonstrate how more than one pointer can refer to a single location in memory. This will allow multiple variables to be able to make updates to a single value.                                                                                                                                                                                                                                                                                                                                                                               |               |                 |           |               |                 |    |   |   |
| Problem                        | Professor Bob keeps the “master copy” of everyone’s grades. He also encourages his students to keep track of their grades so they know how they are doing. Finally, the registrar needs to have access to the grades. To make sure that the various versions remain in sync, he distributes pointers instead of copies. The student got an 86 for his grade. Later, the professor noticed a mistake and updates the grade to 89.                                                                                                                                 |               |                 |           |               |                 |    |   |   |
| Solution                       | In this example, there are three variables: an integer representing the professor’s master copy of the grade (gradeProf), a pointer representing the student’s ability to access the grade (pGradeStudent), and a pointer representing the registrar’s copy of the grades (pGradeRegistrar).                                                                                                                                                                                                                                                                     |               |                 |           |               |                 |    |   |   |
|                                | <table><tr><th>gradeProf</th><th>pGradeStudent</th><th>pGradeRegistrar</th></tr><tr><td>86</td><td>←</td><td>←</td></tr></table>                                                                                                                                                                                                                                                                                                                                                                                                                                 |               |                 | gradeProf | pGradeStudent | pGradeRegistrar | 86 | ← | ← |
|                                | gradeProf                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | pGradeStudent | pGradeRegistrar |           |               |                 |    |   |   |
| 86                             | ←                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | ←             |                 |           |               |                 |    |   |   |
| The code for this examples is: |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |               |                 |           |               |                 |    |   |   |
|                                | <pre>{     // initial grade for the student     int gradeProf = 86;                                // start with a normal data variable      // two people have access     const int * pGradeStudent  = &amp;gradeProf; // a const so student can’t change     const int * pGradeRegistrar = &amp;gradeProf; // registrar can’t change it either      // professor updates the grade.     gradeProf = 89;      // report the results     cout &lt;&lt; *pGradeRegistrar          &lt;&lt; endl;     cout &lt;&lt; *pGradeStudent          &lt;&lt; endl; }</pre> |               |                 |           |               |                 |    |   |   |
|                                | Observe how neither pGradeRegistrar nor pGradeStudent actually contain a grade; they only contain the address of the professor’s grade. Thus, when gradeProf changes to 89, they will reflect the new value when *pGradeRegistrar and *pGradeStudent are dereferenced. This is one of the advantages of using pointers: sharing.                                                                                                                                                                                                                                 |               |                 |           |               |                 |    |   |   |
| Challenge                      | You may notice how pGradeStudent and pGradeRegistrar both have the const prefix. This guarentees that *pGradeStudent won’t inadvertently change gradeProf. As a challenge, try to change the value with the following code:                                                                                                                                                                                                                                                                                                                                      |               |                 |           |               |                 |    |   |   |
|                                | <pre>*pGradeStudent = 99;                                // Yahoo! Easy ‘A’!</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |               |                 |           |               |                 |    |   |   |
|                                | Now, remove the const prefix from the pGradeStudent declaration. Does the compile error go away?                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |               |                 |           |               |                 |    |   |   |
| See Also                       | The complete solution is available at <a href="#">3-3-pointerToMasterCopy.cpp</a> or:                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |               |                 |           |               |                 |    |   |   |
|                                | <pre>/home/cs124/examples/3-3-pointerToMasterCopy.cpp</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |               |                 |           |               |                 |    |   |   |

### Example 3.3 – Point to the “Right” Data

Demo

This example will demonstrate how to conditionally assign a pointer to the “right” data. In this case, the pointer will either refer to one variable or another, depending on a condition.

Problem

Imagine Sam and Sue are on a date and they are trying to figure out who should pay. They decide that whoever has more money in their account should pay. They use that person’s debit card. Recall that the debit card points to the person’s account. In other words, a debit card does not contain actual money. Instead, it carries the account number or a pointer to the account:

Solution

We have two variables here (accountSam and accountSue). We will also have a pointer which will refer to one of the accounts or the other depending on the current balance:

| accountSam | pAccount | accountSue |
|------------|----------|------------|
| 120.01     | →        | 398.21     |

In this case, pAccount will point to Sue’s account because she happens to have more money today.

```
{
    // start off with money in the accounts
    float accountSam = 120.01;    // original account of Sam
    float accountSue = 398.21;    // Sue does a better job saving
    float * pAccount;             // uninitialized pointer

    // who will pay...
    if (accountSam > accountSue)  // warning: do not try this on an actual date
        pAccount = &accountSam;  // the & gets the address of the account.
    else                          // This is much like a debit card number
        pAccount = &accountSue;

    // use the debit card to pay
    float priceDinner = 21.65;    // not an expensive dinner!
    *pAccount -= priceDinner;      // remove price of dinner from Sue's account
    *pAccount -= priceDinner * 0.15; // don't forget the tip

    // report
    cout << accountSam << endl;    // since pAccount points to accountSue,
    cout << accountSue << endl;    // only accountSue will have changed
}
```

Challenge

Just before walking into the restaurant, Sam remembered that he has a saving account that his mother told him was “only for a rainy day.” With Sue pulling out her debit card, the skies are definitely looking cloudy!

```
float accountSamsMother = 562.09;    // for emergency use only!
```

Modify the above program to make it work with a third account.

See Also

The complete solution is available at [3-3-pointerToRightData.cpp](#) or:

```
/home/cs124/examples/3-3-pointerToRightData.cpp
```



## Pointers and functions

In the C programming language (the predecessor to C++), there is no pass-by-reference parameter passing mechanism. As you might imagine, this is quite a handicap! How else can you return more than one variable from a function if you cannot use pass-by-reference? It turns out, however, that this is no handicap at all. In C, we use pass-by-pointer instead.

Passing a pointer as a parameter to a function enables the callee to have the same access to the value as the caller. Any changes made to the dereferenced pointer are reflected in the caller's value. Consider the following example:

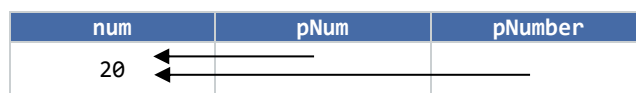
```
/******
 * SET TWENTY
 * Change the value to 20
 *****/
void setTwenty(int * pNumber)           // pass-by-pointer
{
    *pNumber = 20;                     // by changing the dereference value, we
}                                       // change who pNumber points to: num

/******
 * MAIN
 * Simple driver program for setTwenty
 *****/
int main()
{
    // the value to be changed
    int num = 10;                      // the value to be changed
    int * pNum = &num;                 // get the address of num

    // the change is made
    setTwenty(pNum);                   // this could also be: setTwenty(&num);

    // display the results
    cout << num << endl;               // num == 20 due to setTwenty()
    return 0;
}
```

How does this work? Consider the following diagram:



The variable `num` in `main()` starts at the value 10. Also in `main()` is `pNum` which points to `num`. This means that any change made to `*pNum` will be reflected in `num`. When the function `setTwenty()` is called, the parameter `pNumber` is passed. This variable gets initialized with the same address that `pNum` sent. This means that both `pNum` and `pNumber` contain the same address. Thus, any change made to `*pNumber` will be the same as making a change to `*pNum` and `num`. Therefore, when we set `*pNumber = 20` in `setTwenty()`, it is exactly the same thing as setting `num = 20` in `main()`.



## Pass-by-pointer

Up to this point in time, we have had two ways to pass a parameter: **pass-by-value** which makes a copy of the variable so the callee can't change it and **pass-by-reference** which links the variable from the callee to that of the caller so the callee can change it. Now, with pointers, we can use pass-by-pointer. **Pass-by-pointer** is sending a copy of the address of the caller's variable to the callee. With this pointer, the callee can then make a change that will be reflected in the caller's data. To illustrate this point, consider the following code:

```

/*****
 * FUNCTION
 * Demonstrate pass by value, pass by reference, and pass by pointer
 *****/
void function(int value, int &reference, int * pointer)
{
    cout << "value:      " << value      << " &value:      " << &value      << endl;
    cout << "reference: " << reference << " &reference: " << &reference << endl;
    cout << "*pointer:  " << *pointer << " pointer:    " << pointer    << endl;
}

/*****
 * Just call a function. No big deal really.
 *****/
int main()
{
    int number;
    cout << "Please give me a number: ";
    cin  >> number;
    cout << "number:      " << number
         << "\t&number: " << &number
         << endl << endl;

    function(number /*by value */,
             number /*by reference*/,
             &number /*by pointer */);

    return 0;
}

```

This program is available at [3-3-passByPointer.cpp](#) or `/home/cs124/example/3-3-passByPointer.cpp`. The output of this program is:

```

Please give me a number: 100
number:      100  &number: 0x7fff5fcbf07c

value:       100 &value:      0x7fff5fcbf54c
reference:   100 &reference: 0x7fff5fcbf07c
*pointer:    100 pointer:    0x7fff5fcbf07c

```

Observe the value of `number` in `main()` (100) and the address in `main` (0x7fff5fcbf07c). The first parameter is **pass-by-value**. Here, we would expect the value to be copied which it is. Since pass-by-value creates a new variable that is independent of the caller's variable, we would expect that to be in a different address. Observe how it is. The address of `value` is 0x7fff5fcbf54c, different than that of `number`.

The second parameter is **pass-by-reference**. The claim from Chapter 1.4 was that the caller's variable and the callee's variable are linked. Now we can see that the linking is accomplished by making sure both refer to the same location in memory. In other words, because they have the same address (0x7fff5fcbf07c), any change made to `reference` should also change `number`.

The final parameter is **pass-by-pointer**. A more accurate way of saying this is we are passing a pointer by value (in other words, making a copy of the address). Since the address is duplicated in the pointer variable, it follows that the value of `*pointer` should be the same as that of `number`.

The only difference between pass-by-pointer and pass-by-reference is notational. With pass-by-pointer, we need to use the address-of operator (`&`) when passing the parameter and the dereference operator (`*`) when accessing the value. With pass-by-reference, we use the ampersand (`&`) in the callee's parameter. Aside from these differences, they behave exactly the same. This is why the C programming language does not have pass-by-reference: it doesn't need it!

## Pitfall: Changing pointers

As mentioned above, pass-by-pointer is the same thing as passing an address by value. This means that we can change what the pointer is referring to in the function, but we cannot change the pointer itself. To illustrate, consider the following function:

```

/*****
 * CHANGE POINTER
 * Change what pointer is referring to
 *****/
float *changePointer(float * pointer,    // pass-by-pointer. Initially &num1 or p1
                    float &reference)   // pass-by-reference. Tied to num2
{
    *pointer = -1.0;                   // can change *pointer. This will then change
    pointer = &reference;               // num1 in main().
    // this function's copy of p1 will change. It
    // will not change p1 in main because the
    // address was pass-by-value
    return pointer;                    // send reference's address (the same address
    // as num2) back to the caller.
}

/*****
 * MAIN
 * Simple driver program for changePointer
 *****/
int main()
{
    float num1 = 1.0;
    float num2 = 2.0;

    float *p1 = &num1;                // p1 gets the address of num1
    float *p2;                         // p2 is initially uninitialized

    p2 = changePointer(p1, num2);       // because a copy of p1 is sent, it does not
    // change. However, the copy's value is
    // returned which will change p2
    assert(*p1 == -1.0);                // changed in changePointer() to -1.0
    assert(p1 == &num1);                // not changed since it was initialized
    assert(p2 == &num2);                // assigned when function returned to the
    // address of num2

    return 0;
}

```

Notice how the values `num1` and `num2`, as well as the pointers `p1` and `p2` are in `main()` while `pointer` and `reference` are in `changePointer()`. Since `reference` is pass-by-reference to `num2`, they share the same slot in memory. The variable `pointer`, on the other hand, refers to `num1` by pass-by-pointer.

| num1 | p1 | pointer | num2/reference | p2 |
|------|----|---------|----------------|----|
| 1.0  | ←  | ←       | 1.0 ←          | ←  |



If we want to change a pointer parameter in a function (not what it points to), we have three options:

- Return the value and have the caller make the assignment ( `float * change();` )
- Pass a pointer to a pointer. This is called a handle. ( `void change(float ** handle);` )
- Pass the pointer by reference. ( `void change(float * &pointer);` )

## Arrays are pass-by-pointer

As you may recall, arrays are pointers. Specifically, the array variable points to the first item in the range of memory locations. It thus follows that passing an array as a parameter should be pass-by-pointer. In fact, it is. This is why we said in Chapter 3.0 that passing an array is *like* pass-by-reference. The reason is that it is actually pass-by-pointer. Consider the following example:

```

/*****
 * INITIALIZE ARRAY
 * Set the values of an array
 * to zero
 *****/
void initializeArray(int * array,           // we could say int array[] instead, it
                                     int size) // means basically the same thing
{
    for (int i = 0; i < size; i++) // standard array loop
        array[i] = 0; // use the [] notation even though we
                        // declared it as a pointer
}

/*****
 * MAIN
 * Simple driver program
 *****/
int main()
{
    const int SIZE = 10; // const variables are ALL_CAPS
    int list[SIZE]; // can be declared as a CONST
    assert(SIZE == sizeof(list) / sizeof(*list)); // *list is the same as list[0]

    // call it the normal way
    initializeArray(list, SIZE); // call the function the normal way

    // call it with a pointer
    int *pList = list; // list is a pointer so this is OK
    initializeArray(pList, SIZE); // exactly the same as the first time
                                // we called initializeArray

    return 0;
}

```

The square bracket [] notation (as opposed to the pointer \* notation) is convenient because we can forget we are working with pointers. However, they are just a notational convenience. We can remove them and work with pointers to get a more clear indication of what is going on. This program is available at:

```
/home/cs124/examples/3-3-arrays.cpp
```

### Problem 1

What is the output of the following code?

```
{
    float accountSam = 100.00;
    float accountSue = 200.00;
    float * pAccount1 = &accountSam;
    float * pAccount2 = &accountSue;
    float * pAccount3 = &accountSam;
    float * pAccount4 = &accountSue;

    *pAccount1 += 10.00;
    *pAccount2 *= 2.00;
    *pAccount3 -= 15.00;
    *pAccount4 /= 4.00;

    cout << accountSam << endl;
}
```

Answer:

*Please see page 258 for a hint.*

### Problem 2

What is the output of the following code?

```
{
    int a = 10;
    for (int * b = &a; *b < 12; (*b)++)
        cout << ".";
    cout << endl;
}
```

Answer:

*Please see page 258 for a hint.*

### Problem 3

What is the output of the following code?

```
{
    int a = 16;
    int * b = &a;
    int c = *b;
    a = 42;
    int * d = &c;
    a = *b;
    d = &a;
    *d = 99;
    cout << "*b == " << *b <<
    endl;
}
```

| a | b | c | d |
|---|---|---|---|
|   |   |   |   |
|   |   |   |   |
|   |   |   |   |
|   |   |   |   |
|   |   |   |   |
|   |   |   |   |
|   |   |   |   |

Answer:

**Problem 4**

What is the output of the following code?

```
{
    int    a = 10;
    int * b = &a;

    while (*b > 5)
        (*b)--;

    cout << "Answer: "
         << a
         << endl;
}
```

Answer:

---

Please see page 258 for a hint.

**Problem 5**

What is the output of the following code?

```
void funky(int * a)
{
    *a = 8;
    return;
}

int main()
{
    int b = 9;
    funky(&b);
    cout << b << endl;

    return 0;
}
```

Answer:

---

Please see page 262 for a hint.

## Assignment 3.3

Write a program to ask two people for their account balance. Then, based on who has the most money, all subsequent purchases will be charged to his or her account.

### Example

User input is underlined:

```
What is Sam's balance? 229.12
What is Sue's balance? 241.45
Cost of the date:
    Dinner: 32.19
    Movie: 14.50
    Ice cream: 6.00
Sam's balance: $229.12
Sue's balance: $188.76
```

Note that there is a tab before “Dinner,” “Movie,” and “Ice cream.” There are spaces after the colons.

### Challenge

As a challenge, try to write a function to reduce the value of the debit card according to the cost of the date:

```
void date(float *pAccount);
```

This function will contain the three prompts (Dinner, Movie, and Ice Cream) and reduce the value of pAccount by that amount.

### Assignment

Write the program to store the two account balances. Create a pointer to the account that has the largest balance. Then, for the three item costs on the date, reduce the balance of the appropriate account using the pointer. The testbed is:

```
testBed cs124/assign33 assignment33.cpp
```

Don't forget to submit your assignment with the name “Assignment 33” in the header.

*Please see page 261 for a hint.*